PID Control

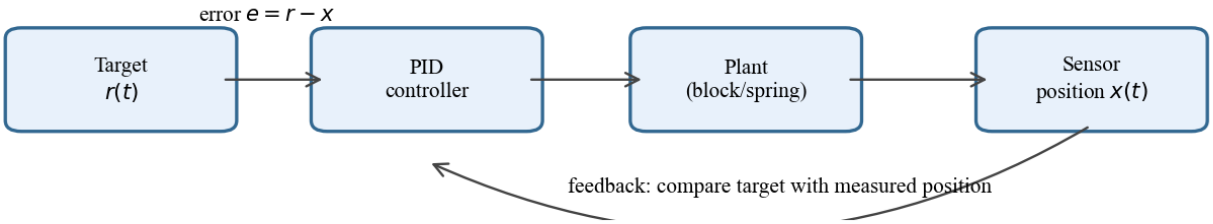
Physical AI Research Intern assignment

1. The basic idea

A controller keeps reducing the gap between a desired value and the value a machine actually produces. It measures the result, calculates the error, and chooses a new input. The loop repeats many times per second.

Everyday example: shower temperature. The target is comfortable water. If the measured water is cold, you open the hot tap more; if it is too hot, you reduce it. A delay between adjustment and feeling the new temperature can cause overshoot, just as a robot can move past its target.

Closed-loop feedback: measure, compare, correct



The sensor closes the loop so the controller can correct the remaining error.

The controller does not need to know the perfect input in advance: feedback lets it correct from what the sensor reports. The quality of the sensor, actuator limits, delays, and controller update rate all affect the result.

2. The PID formula

For a setpoint r(t), measured output x(t), error e(t), and controller output u(t):

$$e(t) = r(t) - x(t)$$

$$u(t) = K_p\, e(t) + K_i \text{ integral}[\, e(\text{tau})\, d\,\text{tau}\,] + K_d\, de(t)/dt$$

For this position controller: r and x are metres, e is metres, and u is force in newtons. Therefore Kp is N/m, Ki is N/(m s), and Kd is N s/m. The integral of error has units m s, while the error rate has units m/s.

P - present error. A proportional push grows with the current gap. It responds quickly, but with a spring load, a nonzero error is needed to maintain a nonzero force.

I - past error. The accumulated error increases the push while an offset persists. It can remove steady-state error, but may keep accumulating while a motor is saturated (integral windup).

D - how error is changing. Derivative action adds damping when motion is approaching the target quickly. It is sensitive to noisy measurements. The implementation uses derivative on measurement to avoid a setpoint kick.

Small-step computer version. With sample interval dt: e[k]=r[k]-x[k]; I[k]=I[k-1]+e[k]dt; D[k]=(e[k]-e[k-1])/dt; u[k]=Kp e[k]+Ki I[k]+Kd D[k]. In the code, D is instead -change(measurement)/dt, which is equivalent for a fixed target.

3. Test system: block, spring and damper

The assignment specifies a 1 kg block attached to a spring and damper. Let x be position in metres and u be applied force in newtons. The damper resists velocity; the spring pulls toward zero.

$$m\,x'' + c\,x' + k\,x = u$$

$$x'' = (u - c\,x' - k\,x)/m = u - 10\,x' - 20\,x$$

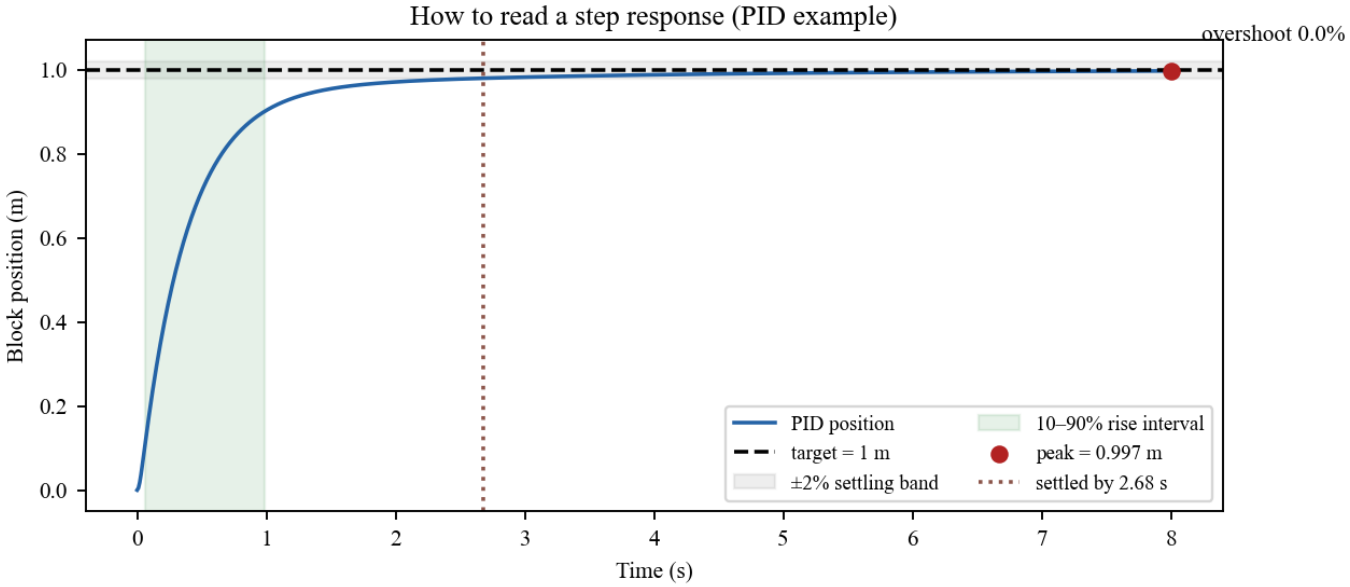
Quantity	Value	Meaning
Mass m	1 kg	Inertia; force produces acceleration
Damping c	10 N s/m	Opposes movement
Spring k	20 N/m	Pulls the block toward x = 0
Target	1 m	Desired block position

Why is a constant push needed? At rest at x=1 m, velocity and acceleration are zero, so the spring force is kx = 20 N. The controller must keep supplying 20 N to balance that pull. P-only control supplies force only when error remains, which explains its steady offset.

Numerical method. The code updates its controller at a fixed dt and holds that force over the next interval. It advances position and velocity with a hand-coded fourth-order Runge-Kutta (RK4) integrator. This is more accurate than a large-step Euler update for the same sample interval.

4. How to judge a result

A step response is the output after the target suddenly changes from 0 m to 1 m. Use defined measurements instead of saying only that one curve looks better.



Metrics use 10–90% rise, peak overshoot, permanent ±2% settling and final 1 s mean error (0.00311 m).

Rise time: time from first crossing 10% of the step to first crossing 90%. **Overshoot:** peak above target divided by step size, as a percent. **Settling time:** time after which the response stays inside a permanent +/-2% target band. **Steady-state error:** target minus the mean output over the final second. If a response never reaches a threshold or settles during the 8 s simulation, it is reported as not reached, not as zero.

These definitions make controller comparisons reproducible. Different applications may choose a different tolerance band or rise-time threshold, so state the definitions alongside the numbers.

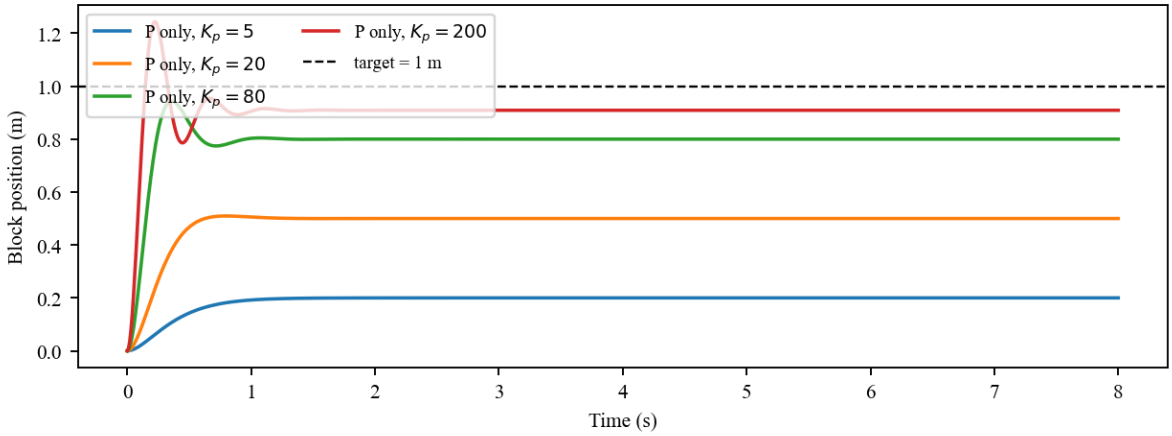
5. Each part alone: P, then I, then D

P-only test. The plot tries four proportional strengths. For a held target r=1 m, steady state has zero velocity and acceleration. Force balance gives u=kx. P control gives u=Kp(1-x), hence:

$$k\,x = Kp(1-x) \;\rightarrow\; x_{ss} = Kp/(Kp+k), \quad e_{ss} = k/(Kp+k)$$

Kp (N/m)	x_ss (m)	error (m)
5	0.2000	0.8000
20	0.5000	0.5000
80	0.8000	0.2000
200	0.9091	0.0909

P-only control: stronger gains reduce, but do not remove, offset

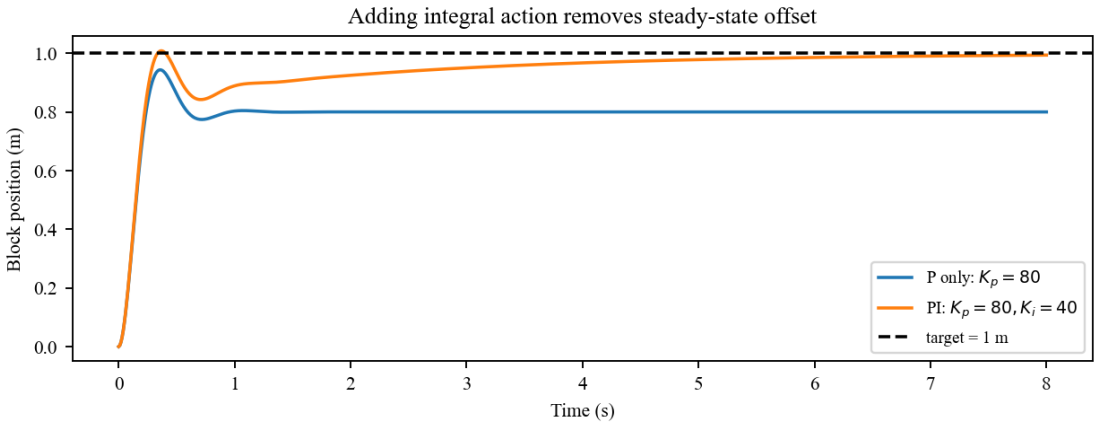


For each finite P gain, the spring pulls the block back and leaves a predictable steady-state error.

As Kp grows, the remaining error shrinks but does not become zero for any finite Kp. The final positions in the plot match the calculation; higher P also makes the transient more aggressive.

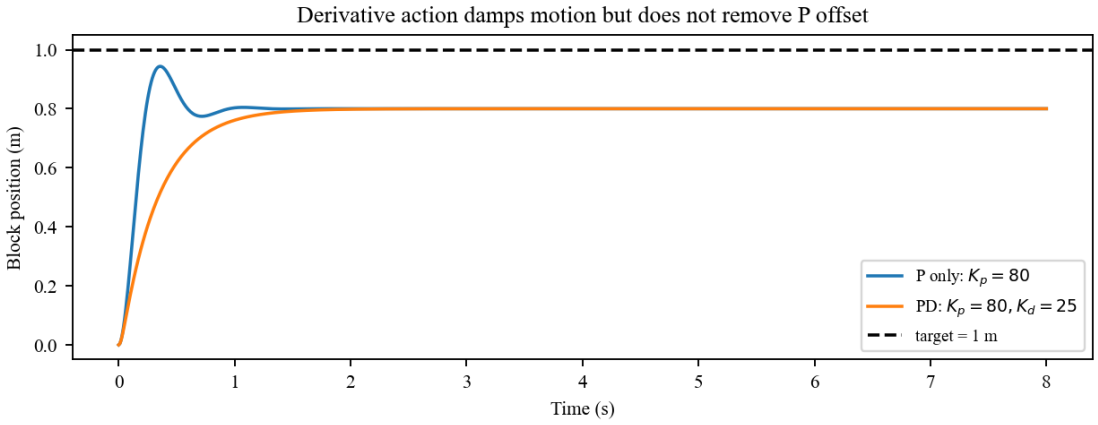
5. Each part alone: adding I and D

Add I. With Kp=80, adding Ki=40 accumulates the small residual error until the controller supplies the spring's required steady force. It removes the P-only offset, but can increase overshoot because accumulated error takes time to unwind.



Integral action supplies the steady force needed to hold the spring at the target.

Add D. With Kp=80, adding Kd=25 reacts against rapid motion. It damps the transient, but does not cancel the spring-load offset; the PD response settles below the 1 m target.

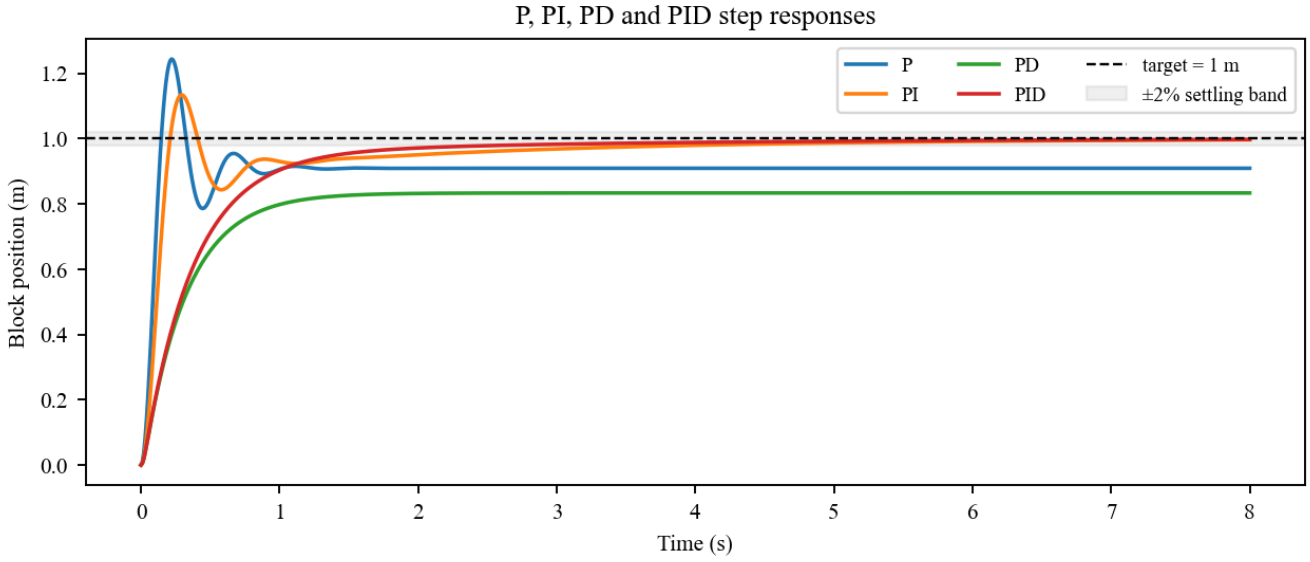


D responds to motion and adds damping; without I, a spring still causes steady-state offset.

In a real sensor, compute and filter D carefully: differentiating sample-to-sample noise can create large force fluctuations.

6. Putting P, I and D together

For a fair comparison, each curve uses a declared gain set on the same plant and target. These are illustrative hand-selected values, not universal optimal gains.



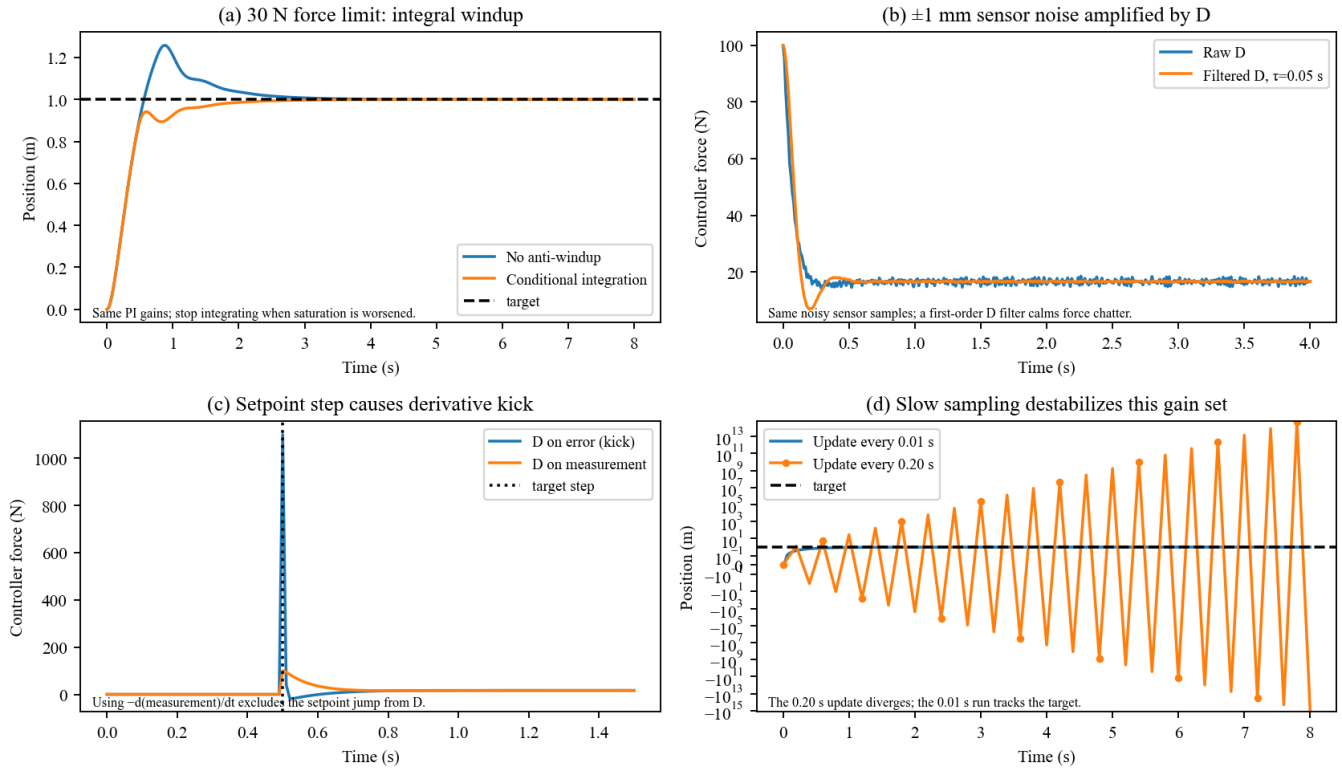
PI/PID can remove spring-load offset; derivative action shapes transient motion.

Controller	Rise (s)	Overshoot (%)	Settling (s)	Final error (m)
P	0.10	24.37	Not reached	0.0909
PI	0.14	13.40	4.05	0.0044
PD	Not reached	0.00	Not reached	0.1667
PID	0.92	0.00	2.68	0.0031

Gains (Kp, Ki, Kd): P (200, 0, 0); PI (120, 60, 0); PD (100, 0, 30); PID (100, 40, 30), in the units given earlier. P and PD do not settle within 2% of the target because of the spring offset; PD also never reaches 90% of the 1 m step. In this example the selected PID reaches the band without overshoot, but its rise is slower. These trade-offs depend on the chosen gains.

7. Real-world problems and fixes

Practical PID problems and fixes



The paired traces isolate each issue; the slow sampled loop diverges with these gains.

(a) Saturation and windup. The actuator is limited to +/-30 N, though the spring needs 20 N at the target. With Kp=120 and Ki=200, the no-anti-windup run overshoots by 25.8%; conditional integration avoids the demonstrated overshoot. It stops integrating when the output is saturated and the current error would push farther into saturation.

(b) Noisy sensor. Uniform measurement noise is bounded at +/-0.001 m (+/-1 mm). D amplifies the sample-to-sample changes; a first-order low-pass filter on D reduces the force chatter. Filtering trades noise reduction for some lag.

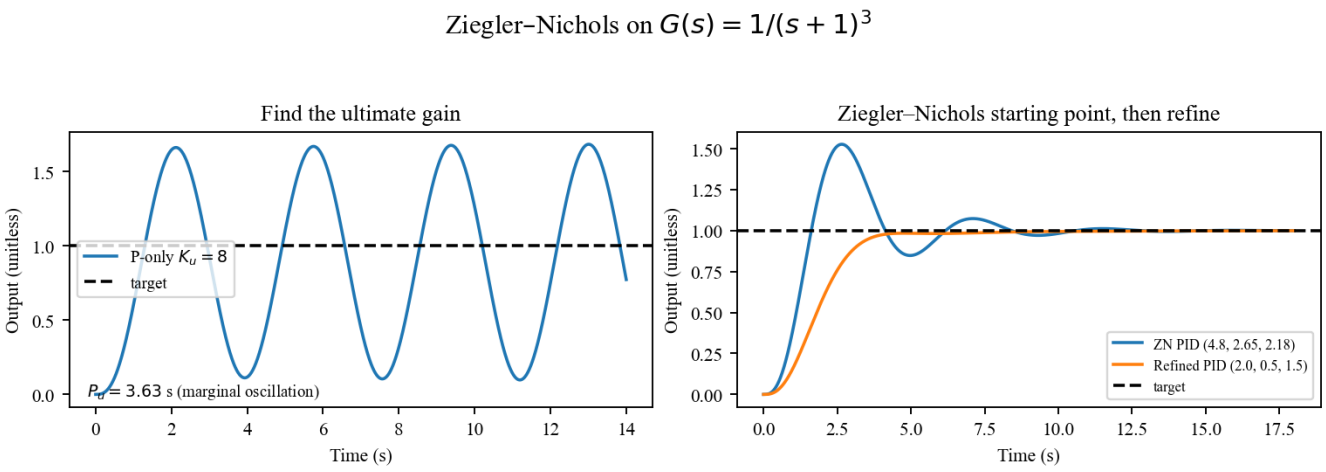
(c) Derivative kick. A target step creates a large derivative if D acts on error. Using -d(measured position)/dt leaves the target out of the derivative term while preserving damping from actual motion.

(d) Slow updates. With this gain set, changing the update interval from 0.01 s to 0.20 s makes the sampled loop unstable; the plotted slow-update response diverges. Sampling delay changes the loop dynamics, so choose an interval fast enough for the plant and retest the closed loop after every rate change.

8. Choosing gains and Ziegler-Nichols

Hand tuning, safely. Start with I=D=0. Raise Kp gradually until response is fast but not dangerously oscillatory. Add D in small steps if overshoot or bouncing is a problem, while checking sensor noise and actuator effort. Add I slowly to remove remaining offset. After each change, compare the same rise, overshoot, settling, error and actuator-limit measures. Test expected loads and limits; reduce gains if oscillations grow.

Second plant. For G(s)=1/(s+1)^3 with unity negative feedback and P-only gain K, the closed-loop characteristic polynomial is (s+1)^3+K. At the stability boundary, K_u=8; the oscillation frequency is sqrt(3) rad/s and P_u=2 pi/sqrt(3)=3.628 s. This theoretical result is checked against the simulated sustained oscillation.



The ZN recipe gives a reproducible initial setting; a gentler tuned set can reduce oscillation.

Rule	Kp	Ti (s)	Td (s)	Equivalent Ki / Kd
P	0.5 Ku = 4.0	-	-	Ki=0, Kd=0
PI	0.45 Ku = 3.6	Pu/1.2 = 3.023	-	Ki=1.191, Kd=0
PID	0.6 Ku = 4.8	Pu/2 = 1.814	Pu/8 = 0.453	Ki=2.646, Kd=2.177

The classical ZN PID setting is intentionally a starting point: on this simulation it overshoots by 52.7% and settles in about 9.91 s. A refined (Kp,Ki,Kd)=(2.0,0.5,1.5) gives 0% overshoot and settles in about 4.22 s, with a slower 2.32 s rise. The safer, less oscillatory trade-off is preferable here.

In the table, Ti is the integral time and Td is the derivative time, both in seconds. To convert the ideal ZN form to the parallel form used by the code, use Ki=Kp/Ti and Kd=Kp*Td.

9. Reusable code and how to reproduce it

The PID class in pid_control.py is reusable: it accepts Kp/Ki/Kd, an optional output limit, derivative-filter time constant, derivative-on-measurement selection, and anti-windup selection. It has no ready-made controller dependency.

Update sequence: validate dt; calculate error; estimate and optionally filter derivative; form a candidate integral; if saturation and the error would deepen it, reject that integration step; sum P/I/D; clamp the command to actuator limits; store current values for the next update.

Plant state: [position, velocity]. The simulated acceleration is (force - 10 velocity - 20 position)/1. RK4 advances that state using the held controller output. Seeded noise makes sensor-noise plots repeatable.

Recreate all graphs and measurements

```
python -m pip install -r requirements.txt
python pid_control.py
python build_report.py
```

The first command installs NumPy, Matplotlib, and ReportLab; ReportLab is only needed for PDF generation. The second regenerates every image in figures/ and metrics.csv. The third assembles those outputs into this PDF. See README.md for output descriptions, assumptions and GitHub publishing steps.

Limitations. This idealized linear model omits friction nonlinearities, actuator delay, backlash, quantization and changing loads. Real robots require safe limits, hardware-aware tuning, and validation on the actual mechanism.

10. Real robots and PID myths

Example	What PID controls
DC motor	Speed loop adjusts voltage/current to reach a requested shaft speed.
Drone	Nested rate/attitude loops correct angular motion; gains and sensing must be tuned for the airframe.
Robot arm	Joint position or velocity control tracks a trajectory while countering load disturbances.
Line-following robot	Image/sensor position error steers the wheels to keep the line centered.
Vehicle cruise control	Throttle or motor torque corrects the difference between requested and measured speed.

Common myths, corrected

Myth: PID always makes a system stable. **Correction:** unsuitable gains, delay, saturation or sampling can cause oscillation or instability.

Myth: one gain set works on every robot. **Correction:** dynamics, units, payload, actuator and sample period matter.

Myth: integral action is always beneficial. **Correction:** it removes persistent error but can wind up at limits and worsen overshoot.

Myth: derivative predicts the future and is always calming. **Correction:** it responds to measured rate; noise and abrupt setpoint changes can create large commands.

Myth: zero steady-state error means a good controller. **Correction:** rise time, overshoot, settling, error, noise and actuator effort all matter.

AI-use disclosure. AI assistance was used to organize explanations, draft documentation, and help check the derivations and simulation approach. The included code generates the reported results; rerun it and verify any gain changes before relying on them.